

## Chapitre 2

# Analyse et Spécification des Besoins

### 2.1 Introduction

Ce chapitre présente une analyse approfondie des besoins fonctionnels et non fonctionnels de la plateforme Bander-Sneakers. Il identifie les acteurs du système, détaille les exigences techniques et modélise l'architecture globale à travers divers diagrammes UML. Cette phase d'analyse constitue le fondement sur lequel repose l'ensemble du développement.

### 2.2 Identification des Acteurs

Le système Bander-Sneakers implique quatre acteurs principaux, chacun ayant des rôles et des interactions spécifiques avec la plateforme.

| Acteur                | Rôle et Permissions                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Visiteur</b>       | <ul style="list-style-type: none"> <li>— Consulter le catalogue de sneakers</li> <li>— Rechercher et filtrer les produits</li> <li>— Consulter les détails des produits</li> <li>— S’inscrire pour devenir client</li> </ul>                                                                                                                                                                                                                        |
| <b>Client</b>         | <ul style="list-style-type: none"> <li>— Toutes les permissions du visiteur</li> <li>— Ajouter des produits au panier et passer commande</li> <li>— Gérer son profil et consulter l’historique des achats</li> <li>— Publier des annonces sur le marché 2nd Hand</li> <li>— Accumuler des points de fidélité</li> <li>— Utiliser le chat en direct avec le support</li> <li>— Ajouter des produits à la wishlist</li> </ul>                         |
| <b>Administrateur</b> | <ul style="list-style-type: none"> <li>— Toutes les permissions du client</li> <li>— Gérer les produits (ajout, modification, suppression)</li> <li>— Gérer les utilisateurs et leurs rôles</li> <li>— Modérer les annonces 2nd Hand</li> <li>— Gérer les promotions et les codes promo</li> <li>— Consulter les statistiques et le dashboard</li> <li>— Gérer le programme de fidélité</li> <li>— Répondre aux messages du chat support</li> </ul> |
| <b>Système</b>        | <ul style="list-style-type: none"> <li>— Gérer l’authentification et les sessions</li> <li>— Envoyer des notifications automatiques</li> <li>— Calculer les points de fidélité</li> <li>— Gérer les emails (confirmation, newsletter)</li> <li>— Assurer la persistance des données</li> </ul>                                                                                                                                                      |

TABLE 2.1 – Identification et rôles des acteurs

## 2.3 Glossaire

|                          |                                                                                 |
|--------------------------|---------------------------------------------------------------------------------|
| <b>Sneaker</b>           | Chaussure de sport lifestyle, principal produit vendu sur la plateforme         |
| <b>SKU</b>               | Stock Keeping Unit - Référence unique identifiant un produit                    |
| <b>Wishlist</b>          | Liste de souhaits contenant les produits favoris d’un utilisateur               |
| <b>2nd Hand</b>          | Section marketplace dédiée à la vente de sneakers d’occasion entre utilisateurs |
| <b>Cross-selling</b>     | Recommandation basée sur les achats conjoints d’autres clients                  |
| <b>Wishlist matching</b> | Recommandation basée sur les favoris de l’utilisateur                           |

|             |                                                                         |
|-------------|-------------------------------------------------------------------------|
| <b>RLS</b>  | Row Level Security - Sécurité au niveau des lignes (Supabase)           |
| <b>CRUD</b> | Create, Read, Update, Delete - Opérations de base sur les données       |
| <b>JWT</b>  | JSON Web Token - Standard de token pour l'authentification              |
| <b>SEO</b>  | Search Engine Optimization - Optimisation pour les moteurs de recherche |
| <b>CDN</b>  | Content Delivery Network - Réseau de distribution de contenu            |

## 2.4 Besoins Fonctionnels

Les besoins fonctionnels sont organisés par modules métier pour une meilleure lisibilité.

### 2.4.1 Module Authentification et Gestion Utilisateur

| ID         | Besoin fonctionnel                                                                          |
|------------|---------------------------------------------------------------------------------------------|
| BF-AUTH-01 | L'utilisateur peut créer un compte avec email, mot de passe, nom et informations de contact |
| BF-AUTH-02 | Le système doit valider l'unicité de l'email et la robustesse du mot de passe               |
| BF-AUTH-03 | L'utilisateur peut se connecter avec ses identifiants                                       |
| BF-AUTH-04 | Le système doit maintenir une session sécurisée après connexion                             |
| BF-AUTH-05 | L'utilisateur peut se déconnecter à tout moment                                             |
| BF-AUTH-06 | L'utilisateur peut réinitialiser son mot de passe via email                                 |
| BF-AUTH-07 | L'utilisateur peut modifier ses informations personnelles (adresse, téléphone)              |
| BF-AUTH-08 | L'utilisateur peut changer son mot de passe depuis son profil                               |
| BF-AUTH-09 | Le système doit gérer les rôles (client/staff) avec privilèges distincts                    |

TABLE 2.2 – Besoins fonctionnels - Module Authentification

### 2.4.2 Module Catalogue et Produits

| ID        | Besoin fonctionnel                                                                   |
|-----------|--------------------------------------------------------------------------------------|
| BF-CAT-01 | Le système doit afficher un catalogue paginé de sneakers                             |
| BF-CAT-02 | L'utilisateur peut filtrer par marque, catégorie, genre, prix, taille                |
| BF-CAT-03 | L'utilisateur peut effectuer une recherche textuelle sur nom, marque, description    |
| BF-CAT-04 | Le système doit proposer des suggestions de recherche dynamiques                     |
| BF-CAT-05 | L'utilisateur peut trier par prix (asc/desc), nom, date d'ajout, popularité          |
| BF-CAT-06 | Chaque produit doit afficher : image principale, nom, marque, prix, réduction        |
| BF-CAT-07 | Le système doit afficher un badge promotionnel si prix réduit                        |
| BF-CAT-08 | L'utilisateur peut accéder à une fiche produit détaillée                             |
| BF-CAT-09 | La fiche produit doit afficher : galerie d'images, description, tailles dispo, stock |
| BF-CAT-10 | La fiche produit doit inclure les avis clients avec notation                         |
| BF-CAT-11 | Le système doit afficher des produits similaires recommandés                         |
| BF-CAT-12 | L'utilisateur peut ajouter un produit au panier depuis la fiche                      |
| BF-CAT-13 | L'utilisateur peut ajouter un produit à sa wishlist                                  |

TABLE 2.3 – Besoins fonctionnels - Module Catalogue

### 2.4.3 Module Panier et Commande

| ID         | Besoin fonctionnel                                                              |
|------------|---------------------------------------------------------------------------------|
| BF-CART-01 | L'utilisateur peut ajouter des produits à son panier (avec taille)              |
| BF-CART-02 | Le système doit vérifier la disponibilité du stock avant ajout                  |
| BF-CART-03 | L'utilisateur peut modifier la quantité d'un article dans le panier             |
| BF-CART-04 | L'utilisateur peut retirer un article du panier                                 |
| BF-CART-05 | Le panier doit afficher le sous-total, frais de port, total TTC                 |
| BF-CART-06 | Le système doit sauvegarder le panier entre sessions (client authentifié)       |
| BF-CART-07 | L'utilisateur peut utiliser des points de fidélité comme réduction              |
| BF-CART-08 | Le processus de checkout doit collecter : adresse, mode livraison, paiement     |
| BF-CART-09 | Le système doit valider la commande et créer une entrée <b>orders</b>           |
| BF-CART-10 | Le système doit déduire automatiquement le stock des articles commandés         |
| BF-CART-11 | Le système doit envoyer un email de confirmation avec détails commande          |
| BF-CART-12 | L'utilisateur doit recevoir une notification de confirmation                    |
| BF-CART-13 | L'utilisateur peut consulter l'historique de ses commandes                      |
| BF-CART-14 | L'utilisateur peut accéder aux détails d'une commande (produits, statut, suivi) |
| BF-CART-15 | L'utilisateur peut télécharger une facture PDF                                  |

TABLE 2.4 – Besoins fonctionnels - Module Panier et Commande

### 2.4.4 Module Système de Recommandation IA

| ID         | Besoin fonctionnel                                                                                |
|------------|---------------------------------------------------------------------------------------------------|
| BF-RECO-01 | Le système doit calculer les recommandations "Les clients ont aussi acheté"                       |
| BF-RECO-02 | La recommandation cross-selling doit analyser les paires d'articles fréquemment achetées ensemble |
| BF-RECO-03 | Le système doit afficher maximum 4 produits recommandés par section                               |
| BF-RECO-04 | Le système doit calculer "Basé sur vos favoris" pour utilisateurs avec wishlist                   |
| BF-RECO-05 | La recommandation wishlist doit identifier les produits similaires (marque, catégorie)            |
| BF-RECO-06 | Les recommandations doivent exclure les produits déjà dans le panier                              |
| BF-RECO-07 | Les recommandations doivent respecter la disponibilité du stock                                   |
| BF-RECO-08 | Le système doit recalculer périodiquement les scores de recommandation                            |

TABLE 2.5 – Besoins fonctionnels - Module Recommandation IA

### 2.4.5 Module Marketplace d’Occasion (2nd Hand)

| ID        | Besoin fonctionnel                                                                       |
|-----------|------------------------------------------------------------------------------------------|
| BF-2ND-01 | L'utilisateur authentifié peut publier une annonce d'occasion                            |
| BF-2ND-02 | Le formulaire doit collecter : titre, description, prix, état, catégorie, marque, taille |
| BF-2ND-03 | L'utilisateur peut uploader jusqu'à 5 images par annonce                                 |
| BF-2ND-04 | Le système doit valider les images (format, taille max 5MB)                              |
| BF-2ND-05 | L'annonce doit avoir un statut (actif, vendu, supprimé, en attente)                      |
| BF-2ND-06 | Le système doit afficher les annonces validées dans le catalogue 2nd Hand                |
| BF-2ND-07 | L'utilisateur peut filtrer/rechercher les annonces d'occasion                            |
| BF-2ND-08 | La fiche annonce doit afficher : images, description, vendeur, localisation              |
| BF-2ND-09 | L'utilisateur peut commenter une annonce                                                 |
| BF-2ND-10 | L'utilisateur peut signaler une annonce suspecte                                         |
| BF-2ND-11 | Le système doit notifier l'admin des signalements                                        |
| BF-2ND-12 | L'admin peut valider/refuser/supprimer une annonce                                       |
| BF-2ND-13 | L'utilisateur peut modifier/supprimer ses propres annonces                               |
| BF-2ND-14 | L'utilisateur peut s'abonner à un vendeur                                                |
| BF-2ND-15 | Le système doit notifier les abonnés lors de nouvelles publications                      |
| BF-2ND-16 | Le profil vendeur doit afficher : annonces, vendus, signalements                         |

TABLE 2.6 – Besoins fonctionnels - Module 2nd Hand

### 2.4.6 Module Programme de Fidélité

| ID        | Besoin fonctionnel                                                               |
|-----------|----------------------------------------------------------------------------------|
| BF-FID-01 | L'utilisateur doit gagner 1 point par euro dépensé lors d'un achat               |
| BF-FID-02 | Le système doit créditer automatiquement les points après confirmation commande  |
| BF-FID-03 | L'utilisateur peut jouer à la roulette journalière (une fois par 24h)            |
| BF-FID-04 | La roulette doit attribuer aléatoirement 0, 5, 10, 15, 20, 25, 50 ou 250 points  |
| BF-FID-05 | Le système doit enregistrer la date/heure du dernier spin                        |
| BF-FID-06 | L'utilisateur peut consulter son solde de points total                           |
| BF-FID-07 | L'utilisateur peut consulter l'historique des transactions de points             |
| BF-FID-08 | L'utilisateur peut utiliser les points comme réduction (conversion configurable) |
| BF-FID-09 | Le système doit débiter les points utilisés lors d'un achat                      |
| BF-FID-10 | Le système doit notifier l'utilisateur des gains de points                       |

TABLE 2.7 – Besoins fonctionnels - Module Fidélité

### 2.4.7 Module Notifications

| ID          | Besoin fonctionnel                                                                |
|-------------|-----------------------------------------------------------------------------------|
| BF-NOTIF-01 | Le système doit notifier l'utilisateur lors d'un changement de prix (wishlist)    |
| BF-NOTIF-02 | Le système doit envoyer un email promotionnel lors d'une baisse de prix           |
| BF-NOTIF-03 | Le système doit notifier l'utilisateur des mises à jour de commande               |
| BF-NOTIF-04 | Le système doit notifier l'utilisateur des réponses chat support                  |
| BF-NOTIF-05 | Le système doit notifier les abonnés de nouvelles annonces 2nd Hand               |
| BF-NOTIF-06 | Le système doit notifier l'utilisateur des gains de points de fidélité            |
| BF-NOTIF-07 | L'utilisateur doit voir un badge de notifications non lues                        |
| BF-NOTIF-08 | L'utilisateur peut consulter le centre de notifications                           |
| BF-NOTIF-09 | L'utilisateur peut marquer une notification comme lue                             |
| BF-NOTIF-10 | L'utilisateur peut supprimer une notification                                     |
| BF-NOTIF-11 | Le système doit horodater chaque notification                                     |
| BF-NOTIF-12 | Le système doit typer les notifications (prix, commande, message, points, report) |

TABLE 2.8 – Besoins fonctionnels - Module Notifications

### 2.4.8 Module Support Client

| ID         | Besoin fonctionnel                                                          |
|------------|-----------------------------------------------------------------------------|
| BF-SUPP-01 | L'utilisateur authentifié peut démarrer une conversation avec le support    |
| BF-SUPP-02 | L'utilisateur peut envoyer des messages texte                               |
| BF-SUPP-03 | Le système doit horodater chaque message                                    |
| BF-SUPP-04 | L'utilisateur doit voir l'historique complet de la conversation             |
| BF-SUPP-05 | L'utilisateur doit recevoir une notification lors d'une réponse admin       |
| BF-SUPP-06 | L'admin peut consulter la liste des conversations actives                   |
| BF-SUPP-07 | L'admin peut répondre à une conversation                                    |
| BF-SUPP-08 | L'admin peut marquer une conversation comme résolue                         |
| BF-SUPP-09 | Le système doit afficher un indicateur de messages non lus                  |
| BF-SUPP-10 | L'interface chat doit se rafraîchir automatiquement (polling ou WebSockets) |

TABLE 2.9 – Besoins fonctionnels - Module Support

### 2.4.9 Module Back-Office Administrateur

| ID          | Besoin fonctionnel                                                                |
|-------------|-----------------------------------------------------------------------------------|
| BF-ADMIN-01 | L'admin doit accéder à un tableau de bord avec statistiques clés                  |
| BF-ADMIN-02 | Le dashboard doit afficher : CA total, nb commandes, nb utilisateurs, nb produits |
| BF-ADMIN-03 | Le dashboard doit afficher des graphiques d'évolution temporelle                  |
| BF-ADMIN-04 | Le dashboard doit afficher les top produits et catégories                         |
| BF-ADMIN-05 | Le dashboard doit afficher les alertes de stock faible                            |
| BF-ADMIN-06 | L'admin peut gérer le catalogue (CRUD produits)                                   |
| BF-ADMIN-07 | L'admin peut uploader plusieurs images par produit                                |
| BF-ADMIN-08 | L'admin peut gérer les stocks par taille                                          |
| BF-ADMIN-09 | L'admin peut appliquer des promotions (prix réduit)                               |
| BF-ADMIN-10 | L'admin peut gérer les marques et catégories                                      |
| BF-ADMIN-11 | L'admin peut consulter la liste des commandes avec filtres                        |
| BF-ADMIN-12 | L'admin peut mettre à jour le statut d'une commande                               |
| BF-ADMIN-13 | L'admin peut consulter les détails d'une commande                                 |
| BF-ADMIN-14 | L'admin peut gérer les utilisateurs (consulter, désactiver)                       |
| BF-ADMIN-15 | L'admin peut modérer les produits d'occasion (valider/refuser)                    |
| BF-ADMIN-16 | L'admin peut modérer les avis clients (supprimer)                                 |
| BF-ADMIN-17 | L'admin peut consulter les signalements et prendre des décisions                  |
| BF-ADMIN-18 | L'admin peut exporter des rapports (CSV, PDF)                                     |

TABLE 2.10 – Besoins fonctionnels - Module Admin

## 2.5 Besoins Non Fonctionnels

### 2.5.1 Sécurité

| ID         | Exigence de sécurité                                                        |
|------------|-----------------------------------------------------------------------------|
| BNF-SEC-01 | Les mots de passe doivent être hachés avec bcrypt (coût minimum 12)         |
| BNF-SEC-02 | Toutes les requêtes SQL doivent utiliser des requêtes préparées (PDO)       |
| BNF-SEC-03 | Les formulaires doivent inclure des tokens CSRF                             |
| BNF-SEC-04 | Les sessions doivent être sécurisées (httpOnly, secure en production)       |
| BNF-SEC-05 | Les uploads de fichiers doivent être validés (type MIME, taille, extension) |
| BNF-SEC-06 | Les entrées utilisateurs doivent être échappées contre XSS                  |
| BNF-SEC-07 | Le système doit implémenter un rate limiting sur les actions sensibles      |
| BNF-SEC-08 | HTTPS doit être obligatoire en production                                   |
| BNF-SEC-09 | Les logs doivent tracer les actions sensibles (connexion, achat, admin)     |
| BNF-SEC-10 | Le système doit respecter le principe du moindre privilège                  |

TABLE 2.11 – Besoins non fonctionnels - Sécurité

### 2.5.2 Performance

| ID          | Exigence de performance                                                    |
|-------------|----------------------------------------------------------------------------|
| BNF-PERF-01 | Le temps de chargement d'une page catalogue ne doit pas excéder 2 secondes |
| BNF-PERF-02 | Le temps de réponse d'une recherche doit être inférieur à 1 seconde        |
| BNF-PERF-03 | Les images doivent être optimisées (compression, lazy loading)             |
| BNF-PERF-04 | La pagination doit limiter les résultats à 20 éléments par page            |
| BNF-PERF-05 | Les requêtes SQL complexes doivent être indexées                           |
| BNF-PERF-06 | Le système doit mettre en cache les données statiques (CDN recommandé)     |
| BNF-PERF-07 | Le système doit supporter 100 utilisateurs concurrent sans dégradation     |
| BNF-PERF-08 | Les calculs de recommandation ne doivent pas bloquer le rendu de page      |

TABLE 2.12 – Besoins non fonctionnels - Performance

### 2.5.3 Disponibilité et Fiabilité

| ID          | Exigence de disponibilité                                                   |
|-------------|-----------------------------------------------------------------------------|
| BNF-DISP-01 | Le système doit viser une disponibilité de 99%                              |
| BNF-DISP-02 | Les sauvegardes de la base doivent être automatiques et quotidiennes        |
| BNF-DISP-03 | Le système doit gérer les erreurs avec des messages utilisateurs explicites |
| BNF-DISP-04 | Les transactions critiques (paiement, stock) doivent être atomiques         |
| BNF-DISP-05 | Le système doit logger les erreurs serveur pour investigation               |
| BNF-DISP-06 | Le système doit prévoir un mode dégradé en cas d'indisponibilité partielle  |

TABLE 2.13 – Besoins non fonctionnels - Disponibilité

### 2.5.4 Ergonomie et Accessibilité

| ID          | Exigence d'ergonomie                                                           |
|-------------|--------------------------------------------------------------------------------|
| BNF-ERGO-01 | L'interface doit être responsive (mobile, tablette, desktop)                   |
| BNF-ERGO-02 | Le design doit suivre les principes d'UX (cohérence, feedback, simplicité)     |
| BNF-ERGO-03 | Les formulaires doivent valider les données en temps réel                      |
| BNF-ERGO-04 | Le système doit afficher des messages de confirmation après actions sensibles  |
| BNF-ERGO-05 | L'interface doit respecter WCAG 2.1 niveau AA (contrastes, navigation clavier) |
| BNF-ERGO-06 | Les icônes doivent être accompagnées de textes alternatifs                     |
| BNF-ERGO-07 | Le temps de réaction de l'interface ne doit pas excéder 100ms                  |

TABLE 2.14 – Besoins non fonctionnels - Ergonomie

### 2.5.5 Maintenabilité et Évolutivité

| ID           | Exigence de maintenabilité                                                  |
|--------------|-----------------------------------------------------------------------------|
| BNF-MAINT-01 | Le code doit être commenté et documenté                                     |
| BNF-MAINT-02 | L'architecture doit suivre le pattern MVC                                   |
| BNF-MAINT-03 | Le code doit respecter les standards PSR (PHP)                              |
| BNF-MAINT-04 | Les fonctions doivent être modulaires et réutilisables                      |
| BNF-MAINT-05 | Le système doit utiliser des gestionnaires de dépendances (Composer)        |
| BNF-MAINT-06 | Les configurations doivent être externalisées (fichiers de config)          |
| BNF-MAINT-07 | Le système doit permettre l'ajout de nouvelles fonctionnalités sans refonte |
| BNF-MAINT-08 | La base de données doit être normalisée (3NF minimum)                       |

TABLE 2.15 – Besoins non fonctionnels - Maintenabilité

## 2.6 Diagrammes UML

### 2.6.1 Diagramme de cas d'utilisation global

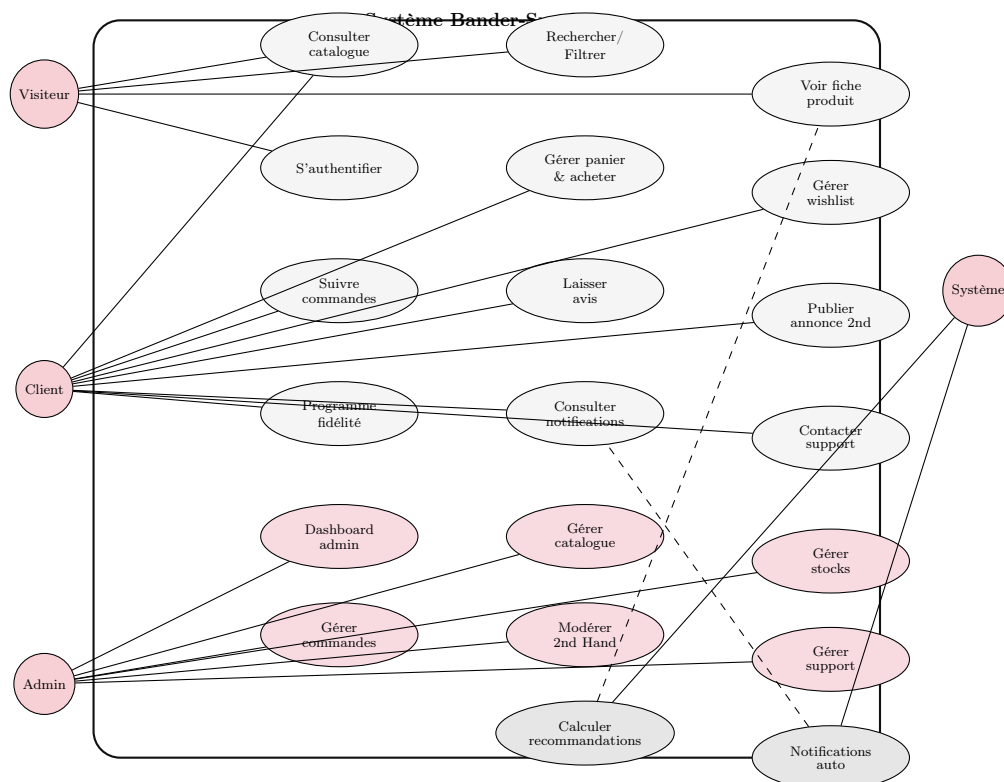


FIGURE 2.1 – Diagramme de cas d'utilisation global Bander-Sneakers

### 2.6.2 Diagramme de classes (extrait)

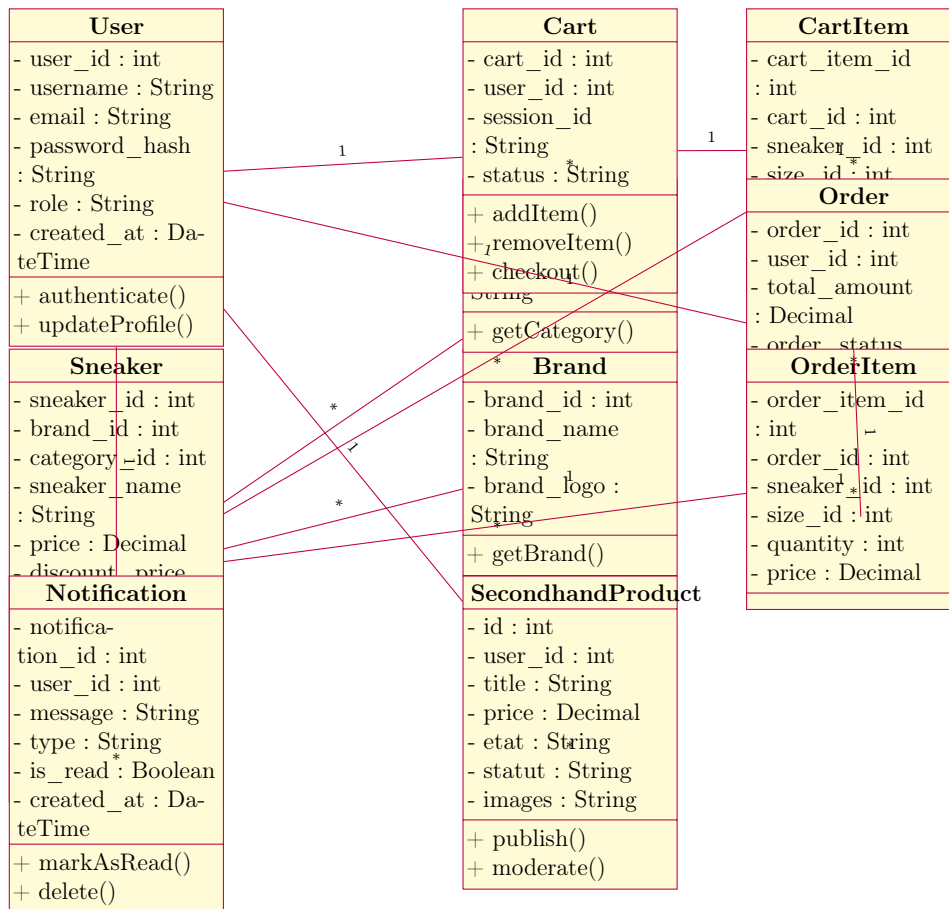


FIGURE 2.2 – Diagramme de classes (extrait principal)

### 2.6.3 Arborescence du projet

L'arborescence suivante présente l'organisation complète des fichiers et dossiers du projet **Bander-Sneakers**. Cette structure reflète une séparation claire entre le front-office (interface utilisateur), le back-office (administration), les assets (ressources statiques) et la logique métier.

```

bander-sneakers/
|-- admin/                                # Admin panel
|   |-- assets/                          # Admin assets (CSS, JS)
|   |   |-- css/
|   |   |   |-- admin.css
|   |   |-- js/
|   |   |   |-- admin.js
|   |
|   |-- includes/                        # Admin shared components
|   |   |-- footer.php
|   |   |-- header.php
|   |   |-- admin-chat.php
|   |
|   |-- admin.php                        # Admin login
|   |-- brands.php                      # Manage brands

```

```

| |-- categories.php           # Manage categories
| |-- index.php               # Admin dashboard
| |-- login.php               # Admin login
| |-- logout.php              # Admin logout
| |-- order-details.php       # Order details
| |-- orders.php              # Manage orders
| |-- product-add.php          # Add products
| |-- product-edit.php         # Edit products
| |-- products.php             # Products list
| |-- profile.php              # Admin profile
| |-- reports.php              # Reports & stats
| |-- reviews.php             # Manage reviews
| |-- secondhand.php           # Manage secondhand products
| |-- secondhand-edit.php      # Edit secondhand products
| |-- settings.php             # Admin settings
| `-- users.php                # Manage users
|
|-- assets/                    # Public assets
| |-- css/
| |   `-- style.css
| |
| |-- js/
| |   `-- (JS files)
| |
| |   `-- images/
| |       |-- brands/         # Brand logos
| |       `-- sneakers/       # Sneaker photos
| |
|-- database/
| |   `-- bander_sneakers.sql  # DB schema
| |
|-- includes/                  # Shared includes
| |-- config.php               # DB config
| |-- functions.php            # Utils
| |-- header.php               # Site header
| |-- footer.php               # Site footer
| |
|-- uploads/                   # User uploads
| |   `-- secondhand/         # Secondhand photos
| |
|-- vendor/                    # External deps
| |-- composer/
| |   `-- autoload.php
| |
| |-- phpmailer/               # Email sending
| |
|-- 2ndhand.php                 # Secondhand listing
|-- 2ndhand-delete.php         # Delete secondhand item
|-- 2ndhand-detail.php         # Secondhand detail
|-- 2ndhand-edit.php           # Edit secondhand item
|-- 2ndhand-post.php           # Post secondhand item
| |
|-- 404.php                     # Error page
|-- about.php                   # About
|-- add-review.php              # Add review
|-- cart.php                    # Cart
|-- cart-add.php                # Add to cart
|-- chat.php                    # Chat page
|-- chat-api.php                # Chat API

```

|                             |                                |
|-----------------------------|--------------------------------|
| -- chat-submit.php          | # Submit message               |
| -- checkout.php             | # Checkout                     |
| -- contact.php              | # Contact                      |
| -- enfants.php              | # Kids section                 |
| -- faq.php                  | # FAQ                          |
| -- femmes.php               | # Section femmes               |
| -- get-notifications.php    | # Récupérer notifications      |
| -- hommes.php               | # Section hommes               |
| -- index.php                | # Accueil                      |
| -- login.php                | # Connexion utilisateur        |
| -- logout.php               | # Déconnexion utilisateur      |
| -- loyalty.php              | # Fidélité                     |
| -- manage-notifications.php | # Gestion notifications        |
| -- newsletter-subscribe.php | # Newsletter                   |
| -- notifications.php        | # Page notifications           |
| -- order-confirmation.php   | # Confirmation commande        |
| -- order-details.php        | # Détails commande             |
| -- privacy-policy.php       | # Politique de confidentialité |
| -- process-orders.php       | # Traitement commandes         |
| -- profile.php              | # Profil utilisateur           |
| -- profile-data.php         | # Données profil               |
| -- register.php             | # Inscription                  |
| -- remove-wishlist.php      | # Retirer wishlist             |
| -- report.php               | # Signalement                  |
| -- returns.php              | # Retours produits             |
| -- search.php               | # Recherche                    |
| -- send-message.php         | # Envoyer un message           |
| -- sneaker.php              | # Fiche produit                |
| -- sneakers.php             | # Catalogue                    |
| -- spin.php                 | # Roulette quotidienne         |
| -- start-conversation.php   | # Nouvelle conversation        |
| -- terms-conditions.php     | # CGU                          |
| -- test.php                 | # Test emails                  |
| -- wishlist.php             | # Wishlist                     |
| -- wishlist-add.php         | # Ajouter wishlist             |
| -- wishlist-remove.php      | # Retirer wishlist             |

### 2.6.4 Description des dossiers principaux

| Dossier/Fichier | Description                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| admin/          | Back-office complet avec gestion des produits, utilisateurs, commandes, modération 2nd hand, statistiques et paramètres. Accès restreint aux administrateurs. |
| assets/         | Ressources statiques du front-office : CSS, JavaScript, images (produits, marques, bannières). Organisation par type de ressource.                            |
| includes/       | Fichiers PHP partagés : configuration BDD, fonctions métier, header/footer réutilisables, middleware d'authentification.                                      |
| database/       | Dump SQL complet de la base de données avec structure (20 tables) et données de démonstration.                                                                |
| PHPMailer/      | Bibliothèque pour l'envoi d'emails (notifications commandes, newsletter, alertes prix).                                                                       |
| uploads/        | Stockage des images uploadées par les utilisateurs (produits d'occasion, avatars). Organisation par type.                                                     |
| *.php (racine)  | Pages principales du front-office : catalogue, produit, panier, checkout, profil, 2nd hand, chat, fidélité, etc.                                              |

TABLE 2.16 – Description des dossiers et fichiers principaux

### 2.6.5 Phases du projet

1. **Analyse (2 semaines)** : Étude du marché, analyse des besoins, rédaction du cahier des charges
2. **Conception (3 semaines)** : Modélisation UML, maquettage, architecture technique, schéma BDD
3. **Frontend (4 semaines)** : Développement des interfaces, intégration responsive, animations
4. **Backend (5 semaines)** : Logique métier, APIs, gestion BDD, sécurité, systèmes avancés
5. **Tests (2 semaines)** : Tests unitaires, intégration, sécurité, performance, UX
6. **Déploiement (1 semaine)** : Mise en production, configuration serveur, optimisations finales
7. **Maintenance (14 semaines)** : Support continu, corrections bugs, améliorations, mises à jour

### 2.6.6 Arborescence du site

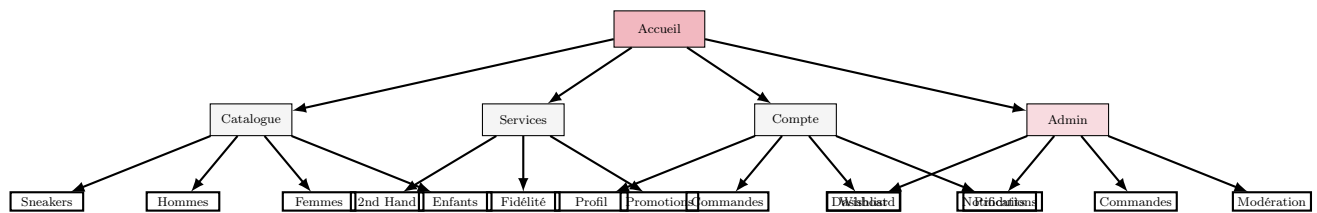


FIGURE 2.3 – Arborescence du site Bander-Sneakers

## 2.7 Architecture technique

### 2.7.1 Architecture logique (MVC)

**Bander-Sneakers** adopte le pattern architectural MVC (Modèle-Vue-Contrôleur) pour séparer les responsabilités :

- **Modèle (Model) :**
  - Gère les données et la logique métier
  - Interaction avec la base de données via PDO
  - Fonctions utilitaires dans `includes/functions.php`
  - Exemples : `getSneakers()`, `addNotification()`, `getLoyaltyPoints()`
- **Vue (View) :**
  - Présente les données à l'utilisateur
  - Templates HTML/CSS/JavaScript
  - Composants réutilisables dans `includes/` (header, footer)
  - Pages principales : `index.php`, `sneakers.php`, `cart.php`
- **Contrôleur (Controller) :**
  - Traite les requêtes utilisateurs
  - Appelle les fonctions du modèle
  - Sélectionne la vue appropriée
  - Scripts de traitement : `cart-add.php`, `process-orders.php`, `login.php`

### 2.7.2 Architecture physique (3-tiers)

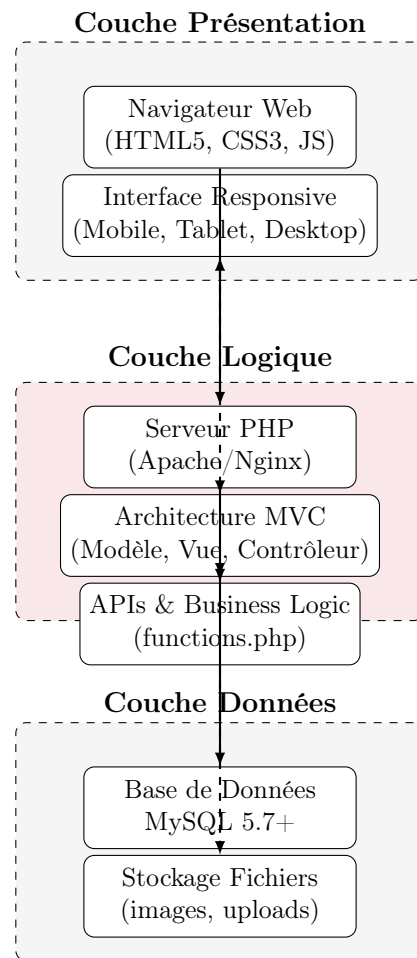


FIGURE 2.4 – Architecture physique 3-tiers

### 2.7.3 Diagramme de déploiement

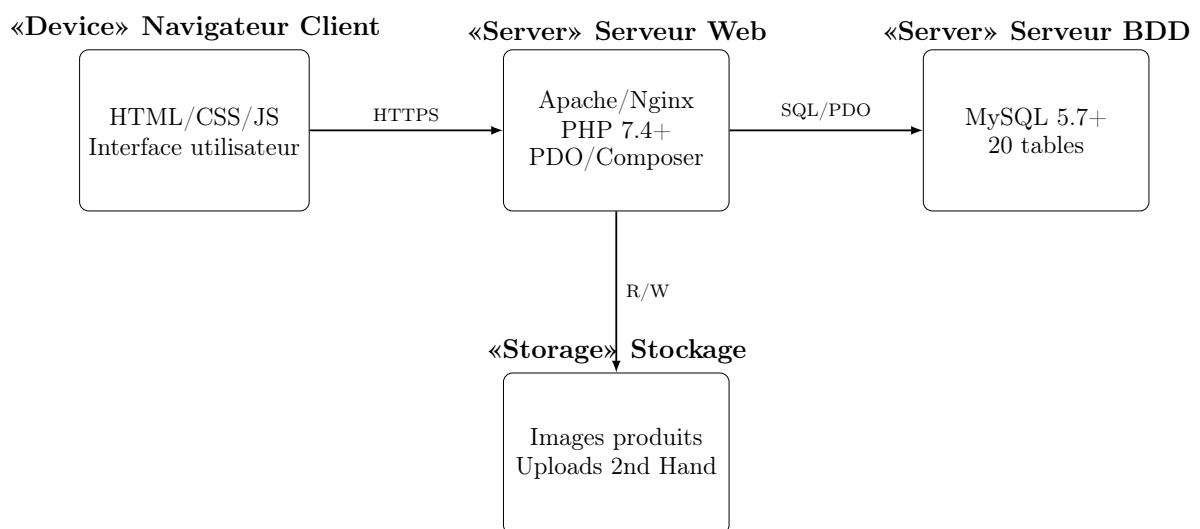


FIGURE 2.5 – Diagramme de déploiement

## 2.8 Schéma de base de données

La base de données de **Bander-Sneakers** comprend 20 tables interconnectées par des clés étrangères, assurant une gestion cohérente des données. Les tables principales incluent **users** (utilisateurs), **sneakers** (produits neufs), **secondhand\_products** (produits d'occasion), **orders** (commandes), et **notifications** (alertes). Les relations clés sont :

- **sneakers** est lié à **brands** et **categories** via **brand\_id** et **category\_id**.
- **sneaker\_sizes** associe **sneakers** à **sizes** pour gérer les stocks par taille.
- **orders** référence **users** pour les commandes, avec **order\_items** détaillant les produits commandés.
- **secondhand\_products** est lié à **users**, **categories**, et **brands**, avec **comments** pour les interactions.
- **notifications** est lié à **users** et peut référencer **sneakers** ou **orders** via **related\_id**.
- **subscriptions** gère les relations de suivi entre utilisateurs.

Un fichier de dump de la base de données a été généré afin de créer une copie complète de sa structure et de son contenu. Cette copie permet de faciliter la sauvegarde, le transfert ou la restauration de la base de données. Le fichier résultant de cette opération est stocké dans le dossier `database`, où il peut être utilisé pour une réimportation ultérieure si nécessaire.



FIGURE 2.6 – Schéma de la base de données de Bander-Sneakers (aussi dans Annexes)

## 2.9 Liste des tables de la base de données

Le tableau suivant répertorie les 20 tables de la base de données, avec une description et les colonnes principales.

| Nom de la table        | Description                                     | Colonnes principales                                                            |
|------------------------|-------------------------------------------------|---------------------------------------------------------------------------------|
| brands                 | Gestion des marques de sneakers                 | brand_id, brand_name, brand_logo, brand_description                             |
| categories             | Catégories de sneakers (ex. Running, Lifestyle) | category_id, category_name, category_description                                |
| users                  | Informations des utilisateurs (clients, admins) | user_id, username, email, password, is_admin, created_at                        |
| loyalty_points         | Points de fidélité des utilisateurs             | id, user_id, points, earned_at                                                  |
| spin_logs              | Historique des spins de la roulette journalière | spin_id, user_id, points_won, spin_date                                         |
| newsletter_subscribers | Abonnés à la newsletter                         | subscriber_id, email, subscribed_at, is_active                                  |
| sizes                  | Tailles disponibles (ex. EU 36)                 | size_id, size_value, size_type                                                  |
| sneakers               | Catalogue des sneakers neuves                   | sneaker_id, sneaker_name, price, brand_id, category_id, gender, primary_image   |
| sneaker_images         | Images des sneakers                             | image_id, sneaker_id, image_url, is_primary                                     |
| sneaker_sizes          | Stock par taille pour chaque sneaker            | sneaker_size_id, sneaker_id, size_id, stock_quantity                            |
| cart                   | Paniers des utilisateurs                        | cart_id, user_id, session_id, status, created_at                                |
| cart_items             | Articles dans les paniers                       | cart_item_id, cart_id, sneaker_id, size_id, quantity                            |
| wishlist               | Liste de souhaits des utilisateurs              | wishlist_id, user_id, sneaker_id, created_at                                    |
| orders                 | Commandes passées                               | order_id, user_id, order_status, total_amount, shipping_address, payment_method |
| order_items            | Détails des articles commandés                  | order_item_id, order_id, sneaker_id, size_id, quantity, price                   |
| reviews                | Avis des utilisateurs sur les sneakers          | review_id, user_id, sneaker_id, rating, review_text                             |
| chat_messages          | Messages du chat en temps réel                  | message_id, user_id, admin_id, message_text, is_admin, created_at               |
| settings               | Paramètres du site (ex. nom, devise)            | setting_id, setting_key, setting_value, setting_description                     |
| secondhand_products    | Sneakers d'occasion mises en vente              | id, user_id, title, price, etat, category_id, brand_id, size, statut            |

| Nom de la table | Description                                  | Colonnes principales                                                          |
|-----------------|----------------------------------------------|-------------------------------------------------------------------------------|
| reports         | Signalements sur produits d'occasion ou avis | report_id, user_id, reported_user_id, type, item_id, reason, etat_signalement |
| comments        | Commentaires sur les produits d'occasion     | comment_id, secondhand_id, user_id, comment_text, created_at                  |
| conversations   | Conversations entre utilisateurs             | conversation_id, user1_id, user2_id, created_at, is_closed                    |
| messages        | Messages dans les conversations              | message_id, conversation_id, sender_id, message_text, sent_at, is_read        |
| notifications   | Notifications utilisateurs (ex. promotions)  | notification_id, user_id, message, is_read, type, related_id                  |
| subscriptions   | Abonnements entre utilisateurs               | subscriber_id, subscribed_to_id, created_at                                   |

TABLE 2.17: Liste des tables de la base de données de Bander-Sneakers

## 2.10 Code source des fonctions principales

### 2.10.1 Fonction getSneakers() - Filtrage dynamique

```
function getSneakers($filters = [], $limit = 0, $offset = 0) {
    $db = getDbConnection();
    $sql = "SELECT s.*, b.brand_name, c.category_name,
        (SELECT image_url FROM sneaker_images
         WHERE sneaker_id = s.sneaker_id AND is_primary = 1
         LIMIT 1) AS primary_image
    FROM sneakers s
    LEFT JOIN brands b ON s.brand_id = b.brand_id
    LEFT JOIN categories c ON s.category_id = c.category_id
    WHERE 1=1";

    // Application dynamique des filtres
    // ... (voir includes/functions.php pour implémentation complète)

    return $results;
}
```

## 2.11 Liens et ressources

- Dépôt GitHub : <https://github.com/binksterrel/bander-sneakers>
- Site démo : <https://bander-sneakers.kesug.com>
- Documentation technique : Disponible dans README.md
- Contact projet :
  - Terrel NUENTSA : 43020094@parisnanterre.fr
  - Mathieu SIEGEL : 43004280@parisnanterre.fr

## 2.12 Glossaire technique

|       |                                                                        |
|-------|------------------------------------------------------------------------|
| API   | Application Programming Interface - Interface de programmation         |
| AJAX  | Asynchronous JavaScript and XML - Communication asynchrone             |
| CRUD  | Create, Read, Update, Delete - Opérations de base sur données          |
| MVC   | Model-View-Controller - Pattern architectural                          |
| ORM   | Object-Relational Mapping - Abstraction base de données                |
| PDO   | PHP Data Objects - Extension d'accès aux données                       |
| RGPD  | Règlement Général sur la Protection des Données                        |
| SEO   | Search Engine Optimization - Optimisation moteurs de recherche         |
| UML   | Unified Modeling Language - Langage de modélisation                    |
| UX/UI | User Experience / User Interface - Expérience et interface utilisateur |